

Reproducing ParticleNet for Jet Tagging

Xinzhou Guo

PHY 506 Computational Physics II

4/29/2026

Motivation

Jets in LHC:

- Sprays of particles from high-energy collisions
- Origin: quarks, gluons, heavy particles (top, W, Z, Higgs)
- Complex internal structure (multi-prong, radiation)
- Large fluctuations due to QCD
- No intrinsic ordering of particles

Goal:

Identify origin particle (top, quark, gluon...)

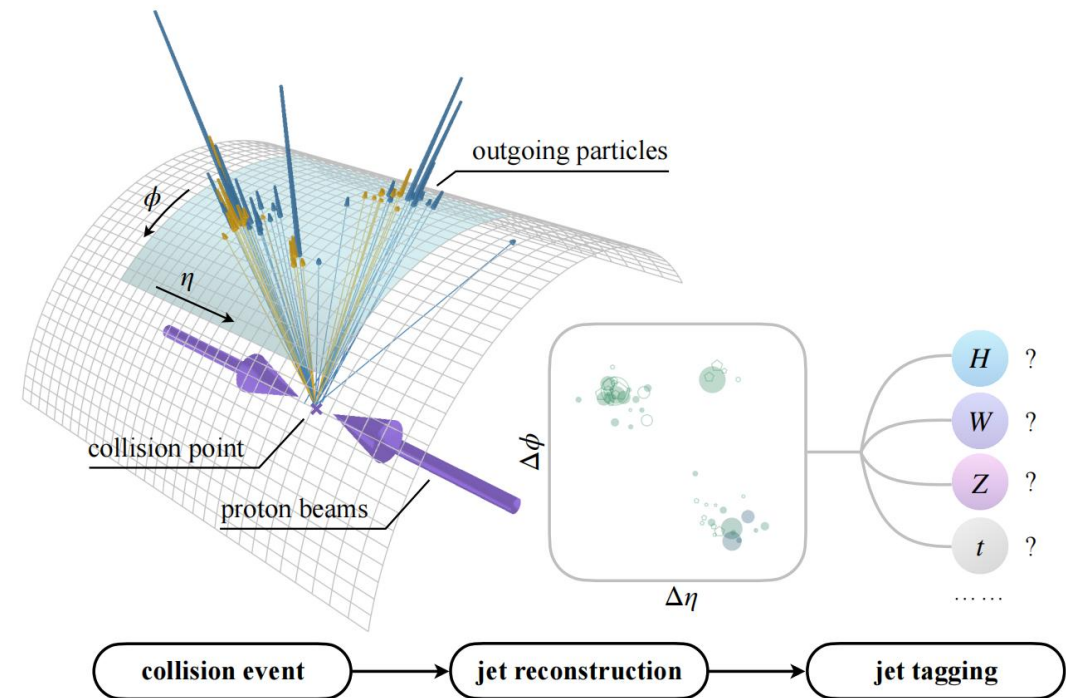


Illustration of jet tagging at the CERN LHC. ([Qu et al, 2022](#))

Motivation

Machine learning approach:

- Uses low-level information (kinematics, tracks, energy deposits)
- Improves discrimination power
- Important for LHC physics (signal vs background)

Previous methods:

- Image-based (CNN), Sequence-based (RNN/1D CNN)
- Both representations impose artificial structure on unordered particles

Solution:

The Particle cloud representation

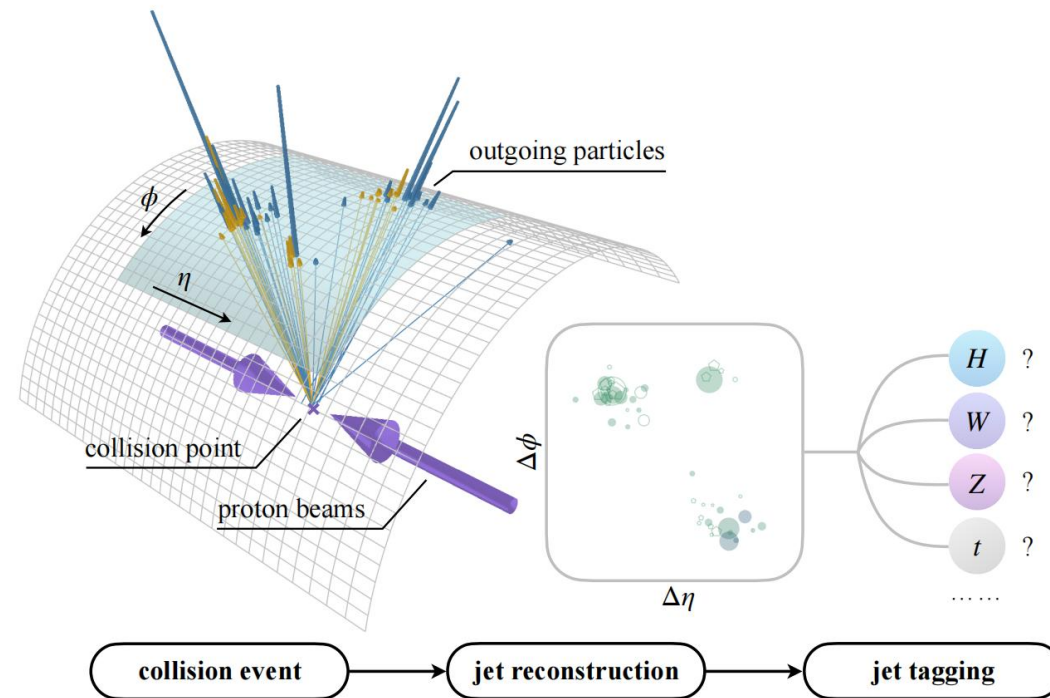


Illustration of jet tagging at the CERN LHC. ([Qu et al, 2022](#))

The Particle Cloud Idea

Particle cloud:

- Treats the jet as an **unordered set** of particles
- Similar to point cloud in computer vision
- Each particle has features like momentum and position (feature vector)
- Particle-level information is **permutation invariant / symmetry**
- Does not depend on the ordering of particles

Alternative Applications:

- The Deep Sets framework ([M. Zaheer et al, 2017](#))
- The Message-passing neural networks (MPNNs) ([I. Henrion et al, 2017](#))

EdgeConv & Latent Graph

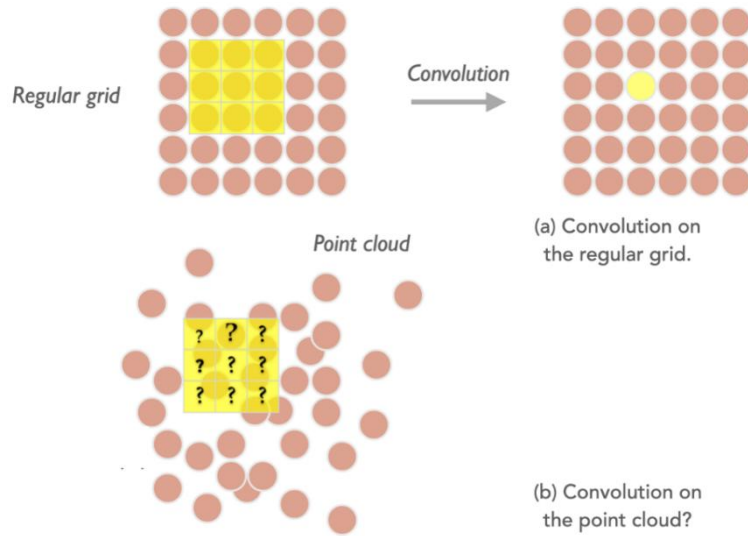
How does ParticleNet work?

1. Construct graph using ***k*-nearest neighbors**
2. Compute **EdgeConv** (local particle relations)
3. Aggregate neighbor information
4. Update particle features
5. Repeat for multiple layers
6. Dynamically update graph (**latent graph**)

EdgeConv & Latent Graph

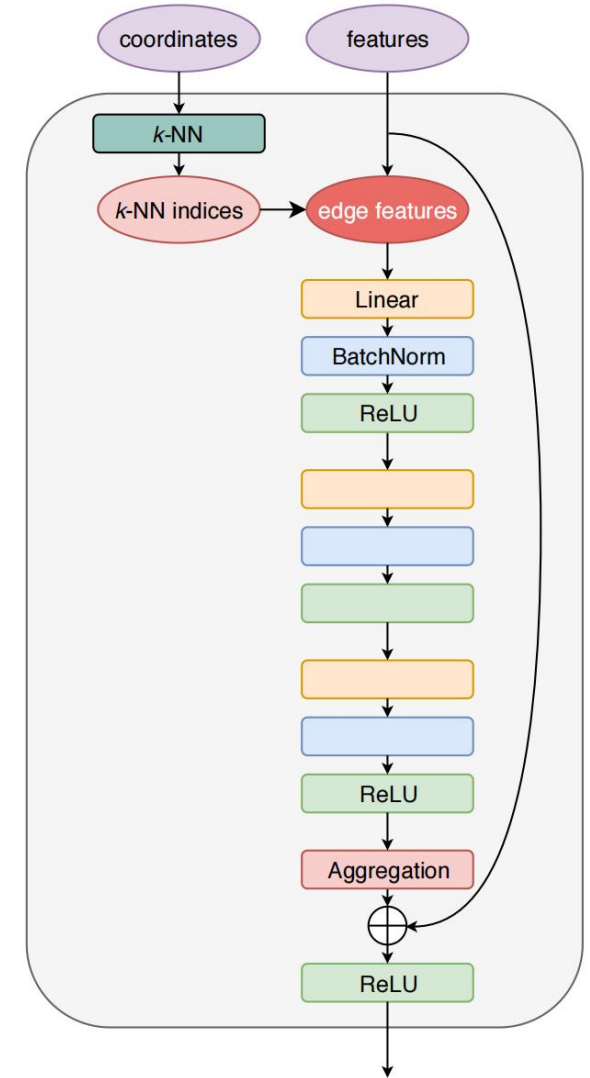
How does ParticleNet work?

- Build graph using k -nearest neighbors
- Apply **EdgeConv** (a graph convolution operation)
- Aggregate neighbor information and update features



The cartoon illustrates the convolutional operation acted on the regular grid and on the point cloud (from [ML4Jets 2018](#)).

The structure of the EdgeConv block.
([Qu & Gouskos, 2020](#))



EdgeConv & Latent Graph

How does ParticleNet work?

EdgeConv:

For each particle i :

- look at its neighbors j
- compute relation: “how is j different from i ?”
- pass that through a neural network
- average over neighbors

It learns:

- local correlations between particles
- how particles cluster and how energy flows within the jet

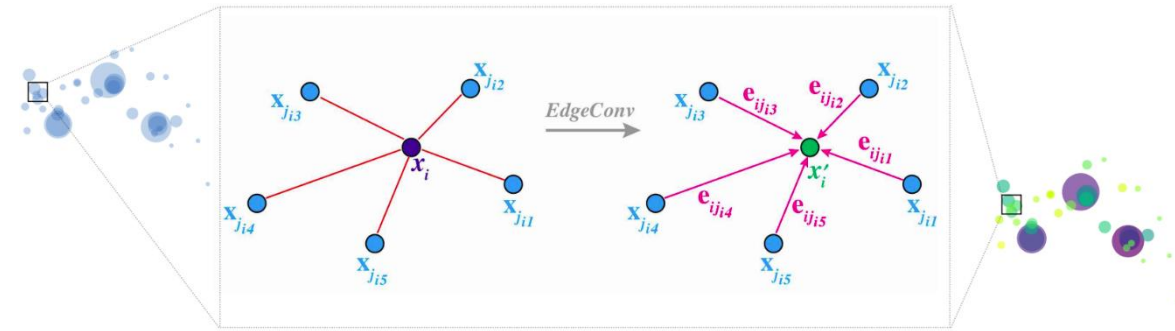


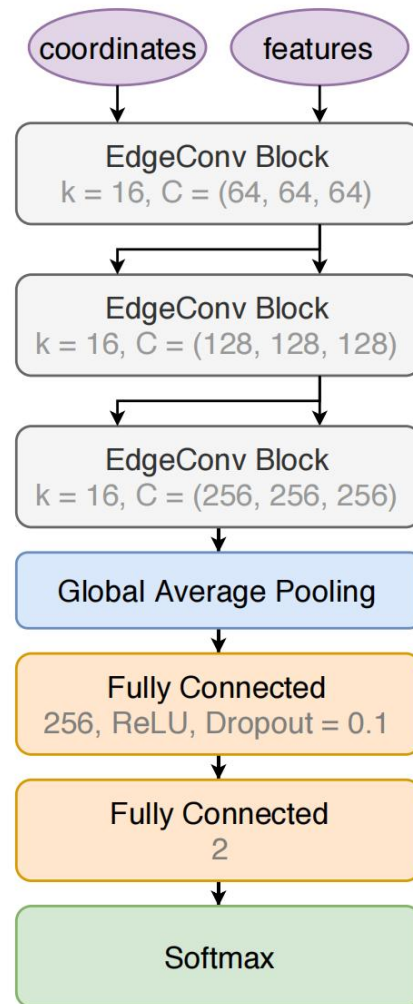
Illustration of the EdgeConv block from [the documentation page](#).

EdgeConv & Latent Graph

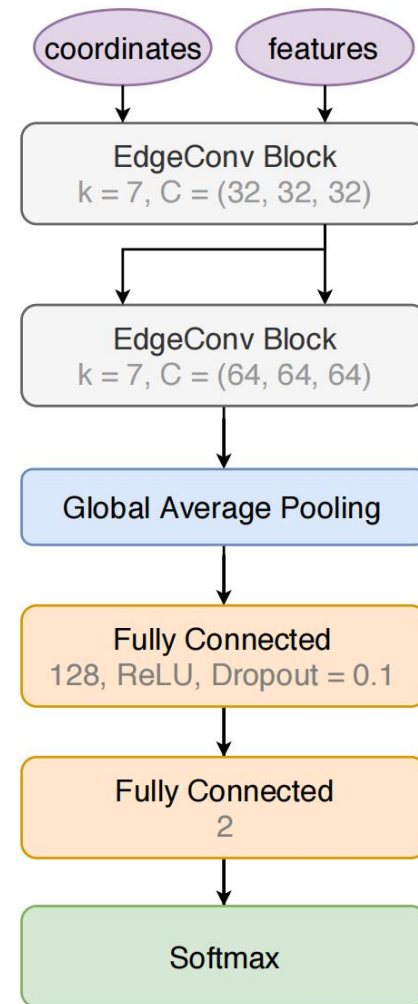
How does ParticleNet work?

- Repeated in multiple layers for complex hierarchical structures
- Graph updated dynamically (**latent graph**)

The architectures of the ParticleNet and the ParticleNet-Lite networks.
([Qu & Gouskos, 2020](#))



(a) ParticleNet



(b) ParticleNet-Lite

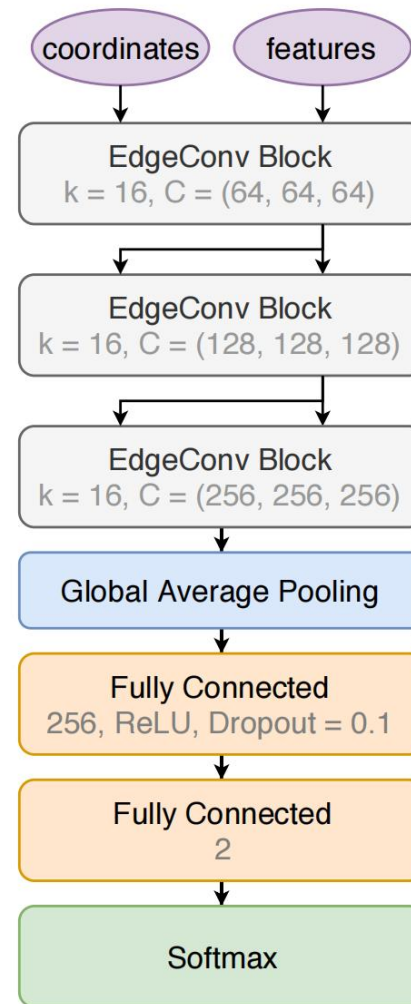
EdgeConv & Latent Graph

How does ParticleNet work?

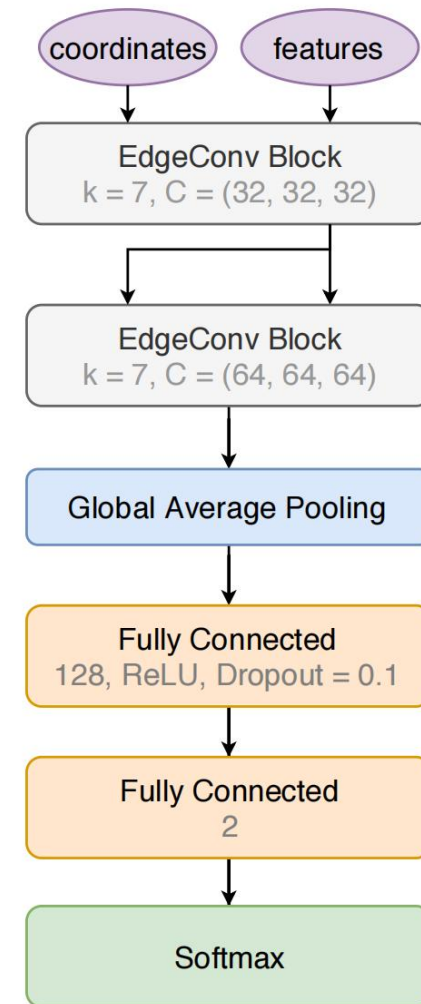
The **latent graph**:

- First layer: neighbors defined using physical coordinates ($\Delta\eta$, $\Delta\phi$)
- After first layer: treat learned features as coordinates and build neighbors again

The architectures of the ParticleNet and the ParticleNet-Lite networks.
([Qu & Gouskos, 2020](#))



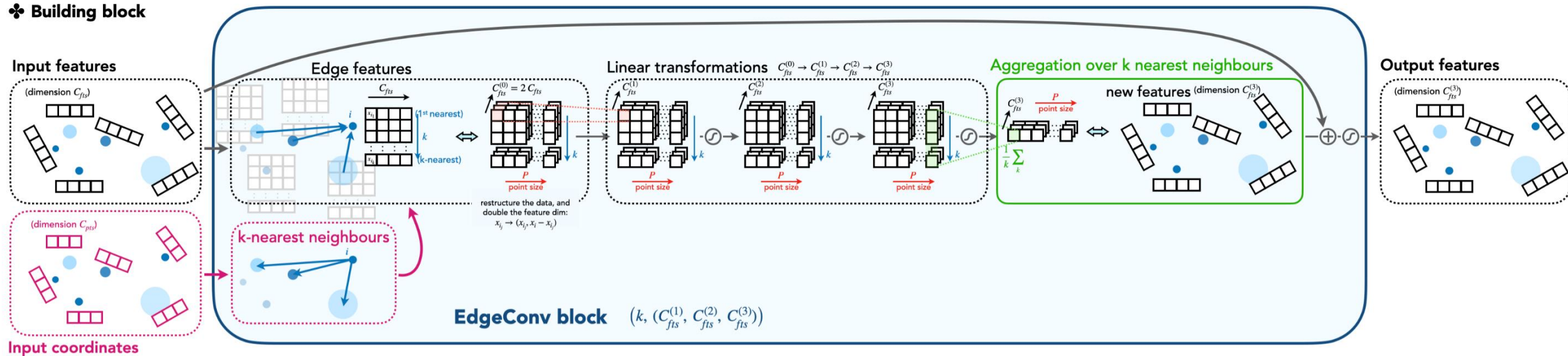
(a) ParticleNet



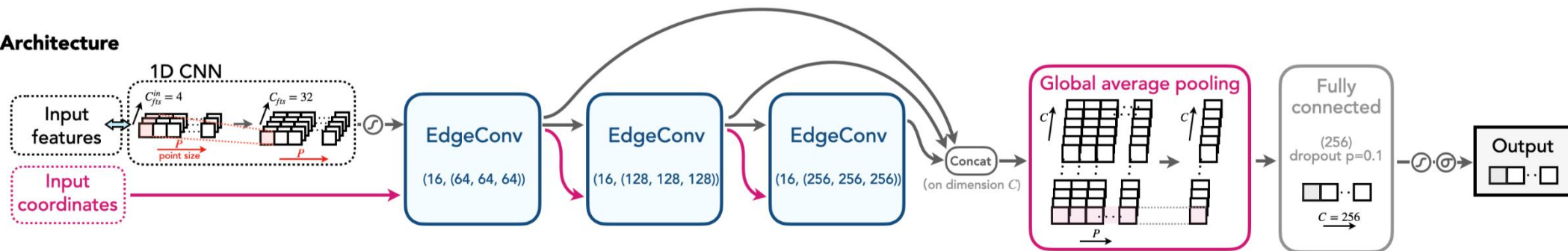
(b) ParticleNet-Lite

ParticleNet Architecture

❖ Building block



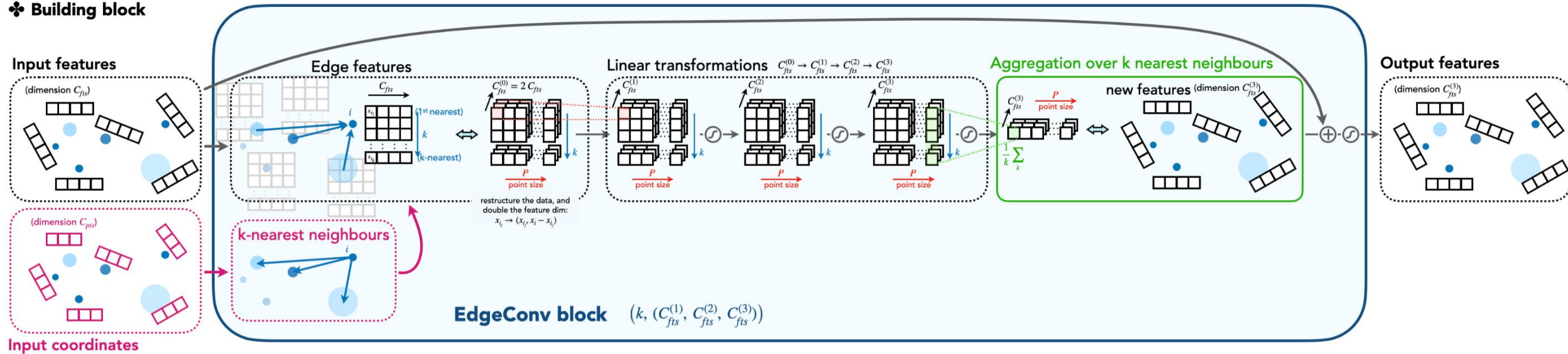
❖ Architecture



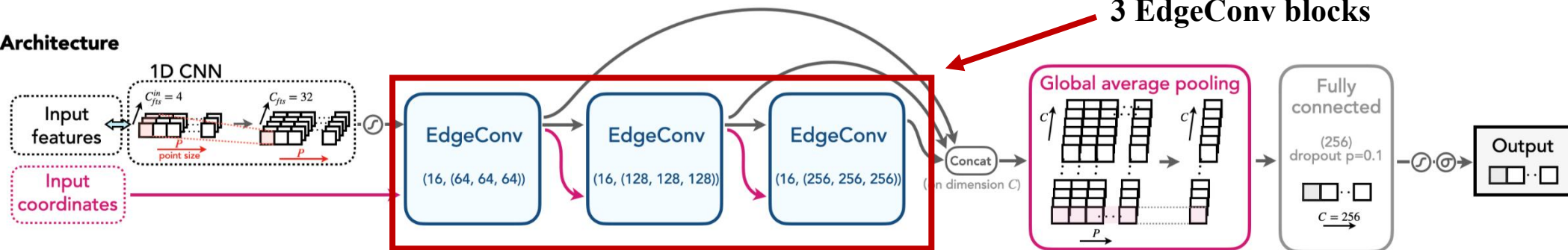
The full architecture of the ParticleNet model from [the documentation page](#).

ParticleNet Architecture

❖ Building block



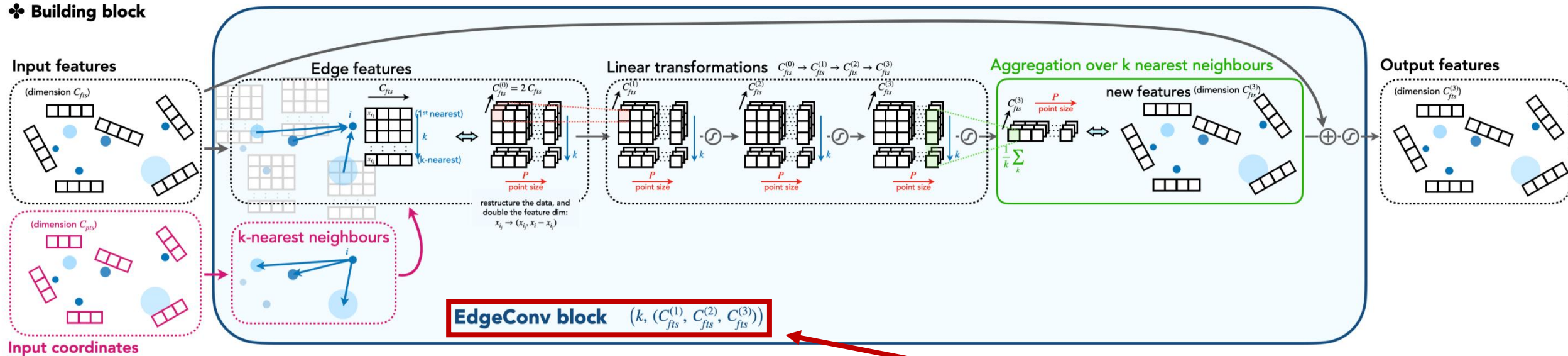
❖ Architecture



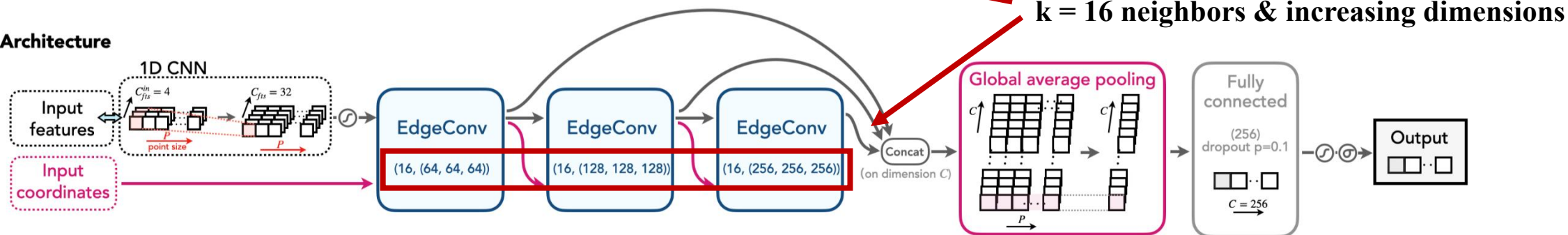
The full architecture of the ParticleNet model from [the documentation page](#).

ParticleNet Architecture

❖ Building block



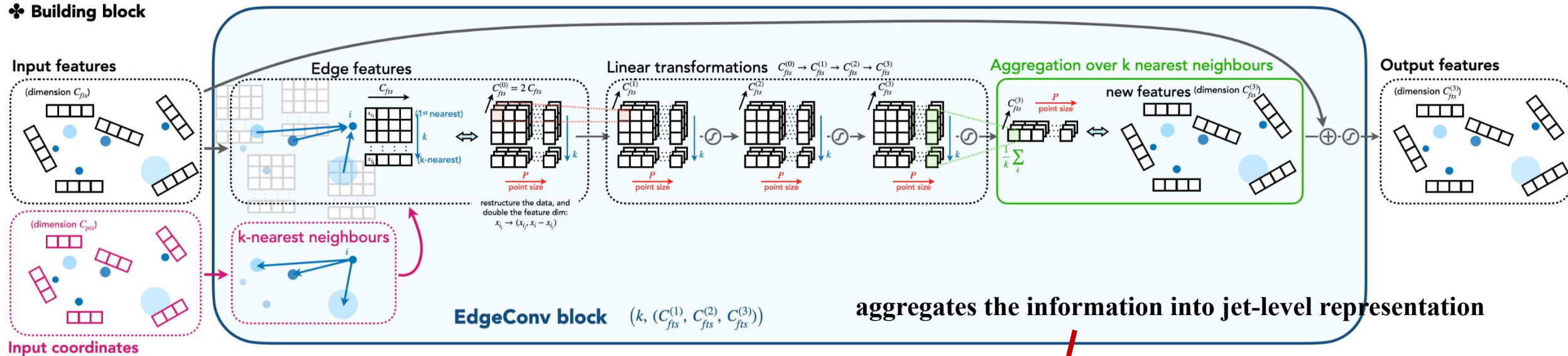
❖ Architecture



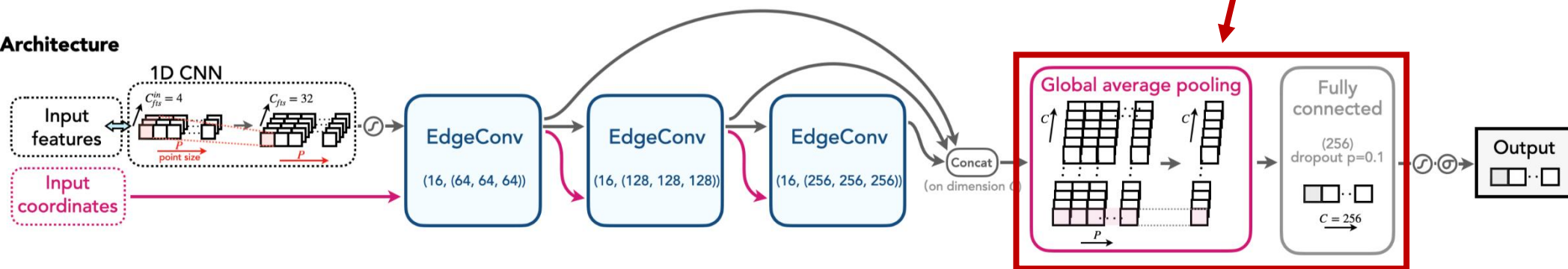
The full architecture of the ParticleNet model from [the documentation page](#).

ParticleNet Architecture

❖ Building block



❖ Architecture



The full architecture of the ParticleNet model from [the documentation page](#).

Evaluation of Models

- Performance Comparison:
 - MLP, DeekAK8 and **ParticleNet**
 - All were trained and evaluated on the same dataset for fair comparison
 - Procedure: Train → Evaluate → Predict → Plot ROC

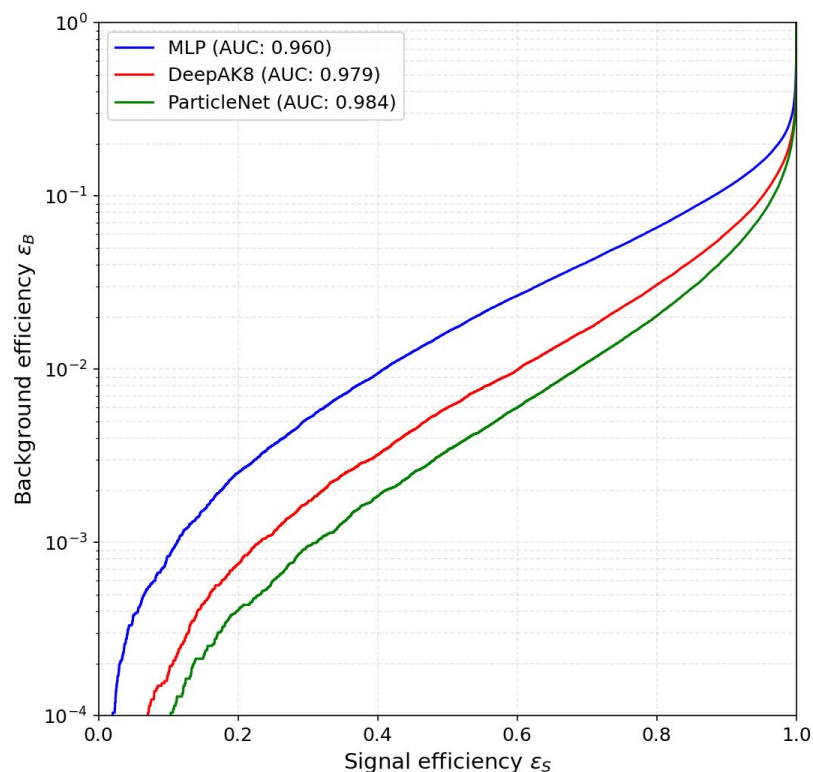
Model	<code>\${PREFIX}</code>	<code>\${MODEL_CONFIG}</code>	<code>\${DATA_CONFIG}</code>	Parameters	Computational complexity
MLP	<code>mlp</code>	<code>mlp_pf.py</code>	<code>pf_features.yaml</code>	739k	0.001 GMac
DeepAK8 (1D CNN)	<code>deepak8</code>	<code>deepak8_pf.py</code>	<code>pf_features.yaml</code>	349k	0.012 GMac
ParticleNet (DGCNN)	<code>particlent</code>	<code>particlenet_pf.py</code>	<code>pf_points_features.yaml</code>	577k	0.441 GMac

The three models used for illustration and comparison, from [the documentation page](#).

Evaluation of Models

The ROC Curve

- Compare the performance of three trained models with the predicted root files.
- Dataset: benchmark of the top tagging task ([CMS Collaboration, 2017](#))



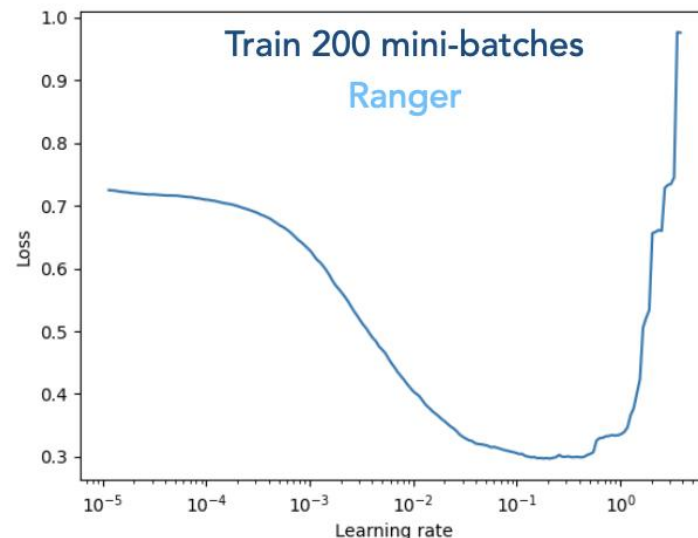
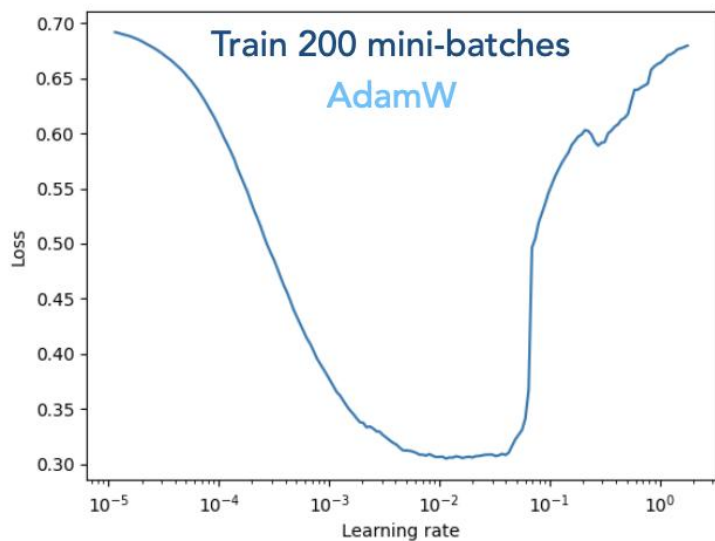
Model	AUC	Accuracy	$1/e^B$ (@ $e^S=0.3$)
MLP	0.960	0.897	193
DeepAK8 (1D CNN)	0.979	0.926	584
ParticleNet (DGCNN)	0.984	0.937	1057

The ROC curve (background vs signal efficiency) for three models, using Ranger and start learning rate of $5e-3$.

Tuning

- The optimizer

- A wiser strategy: avoid the zig-zagging updating route, local minima, adapt different strategies for the fast-changing parameters and the slow ones
- **Adam** (and its several variations): a widely used optimizer
- **Ranger**: recently developed, advanced optimizer, combines RAdam and LookAhead



The results from LR finder by specifying optimizer AdamW (left) and Ranger (right) case, from [the documentation page](#).

Tuning

- The learning rate
 - The most important hyperparameter to choose from before concrete training
 - How fast the parameters changes in one training iteration
 - The Ranger case can choose the range $1e-3$ to $1e-2$
 - The AdamW in general requires a small LR than Ranger

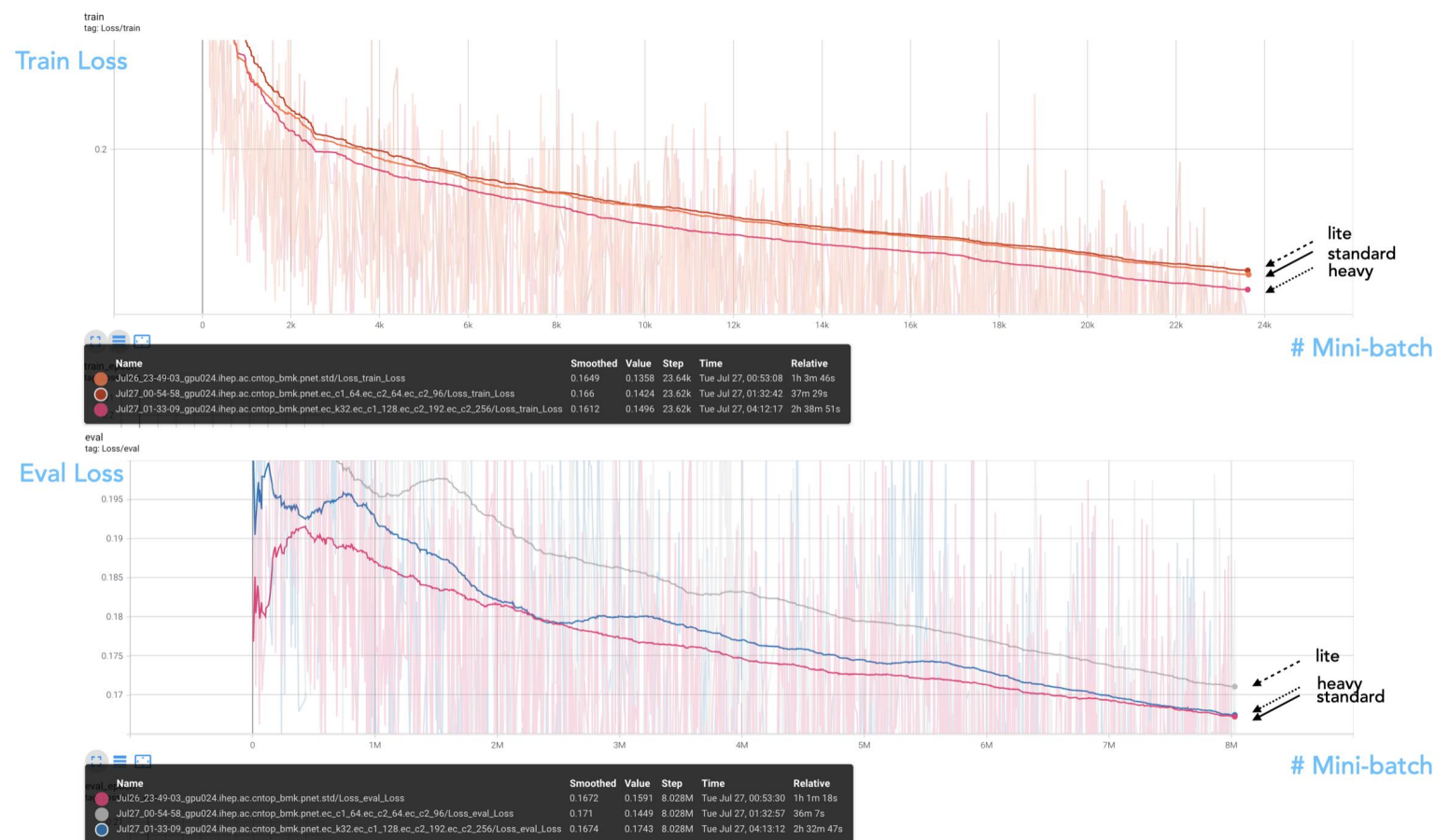
The model size

- A smaller model tends to have unsatisfactory performance:
 - limited ability to learn many local features
- A larger/heavier model may encounter the network "degradation":
 - deeper models have difficulty learning features
 - cause overfitting

Tuning

The model size

- The "heavy" model reaches even smaller training loss: the model does not meet the degradation issue yet
- The evaluation loss is not catching up: showing some degrees of overtraining
- No improvement by moving to a heavy model

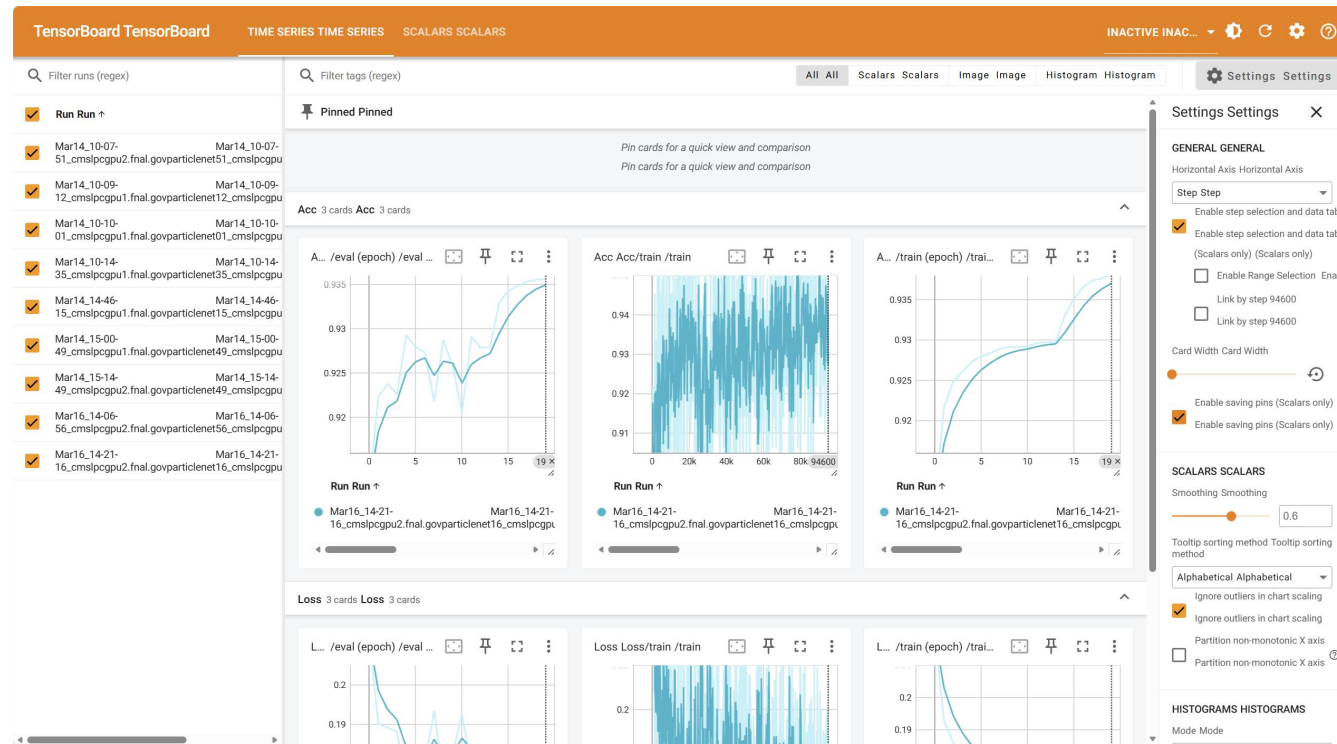


The Tensorboard monitoring plots in the training/evaluation loss, from [the documentation page](#).

Training Visualization

- TensorBoard

- Monitor the full training/evaluation accuracy and the loss for each mini-batch
- Monitor training convergence and detect overfitting



The training results results.

Summary

- Successfully reproduced ParticleNet results
- Verified performance with ROC curves
- Understood graph-based learning for jets
- Demonstrated importance of representation

Future Work

Particle Transformer (GloParT)

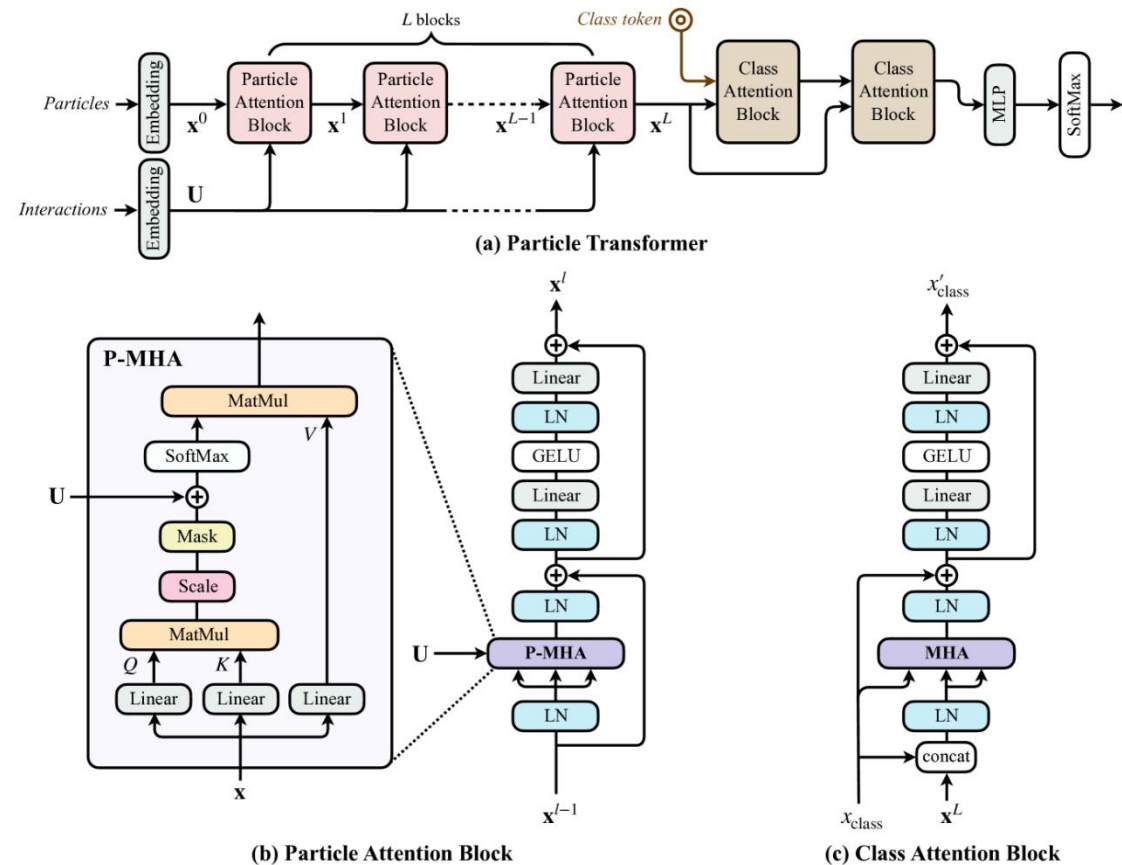
To explore more powerful architectures (transformers) for jet tagging

- Pretraining

- Large dataset (~10M jets, many classes), used as a general feature extractor
- General-purpose jet representation, encodes rich jet features

- Fine-tuning

- Load pretrained GloParT
- Build analysis-specific dataset
- Train a lightweight classifier (MLP)
- Adapting to specific signal and/or background requirements for particular analyses



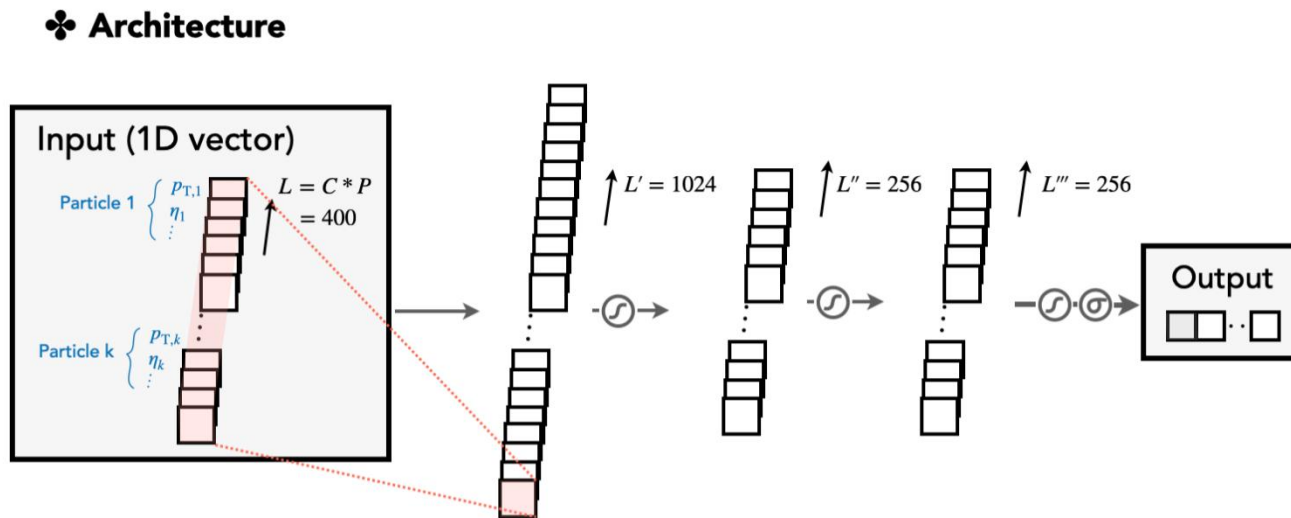
The architecture of (a) Particle Transformer (b) Particle Attention Block (c) Class Attention Block ([Qu et al, 2022](#)).

References

- H. Qu and L. Gouskos, “*Jet Tagging via Particle Clouds*”, Phys. Rev. D 101, 056019 (2020). [arXiv:1902.08570](https://arxiv.org/abs/1902.08570)
- CMS Machine Learning Group, “ParticleNet Documentation”, <https://cms-mml.github.io/documentation/inference/particlenet.html>
- Y. Wang et al., “Dynamic Graph CNN for Learning on Point Clouds”, ACM TOG 2019. [arXiv:1801.07829](https://arxiv.org/abs/1801.07829)
- M. Zaheer et al., “Deep Sets”, NeurIPS 2017. [arXiv:1703.06114](https://arxiv.org/abs/1703.06114)
- H. Qu et al., “Particle Transformer for Jet Tagging”, JHEP 2022. [arXiv:2202.03772](https://arxiv.org/abs/2202.03772)
- CMS Collaboration, “DeepAK8: A Deep Learning-based Jet Tagger”, CMS-DP-2017-049
- CMS Collaboration, “Identification of heavy-flavour jets with the CMS detector in pp collisions at 13 TeV”, JINST 13 (2018) P05011. [arXiv:1707.08966](https://arxiv.org/abs/1707.08966)

Supplementary

- MLP Architecture

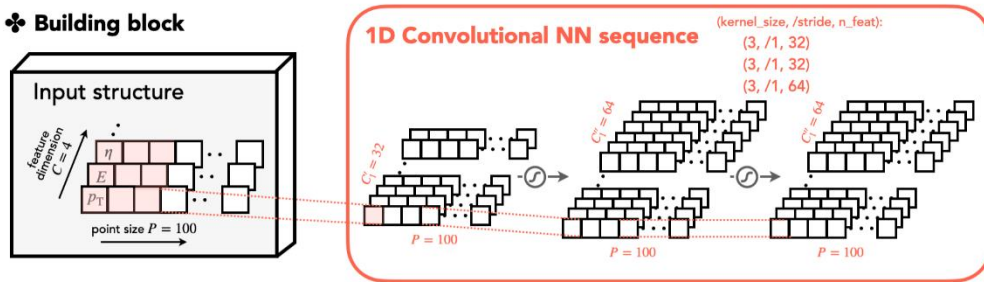


The full architecture of the proof-of-concept multi-layer perceptron model from [the documentation page](#).

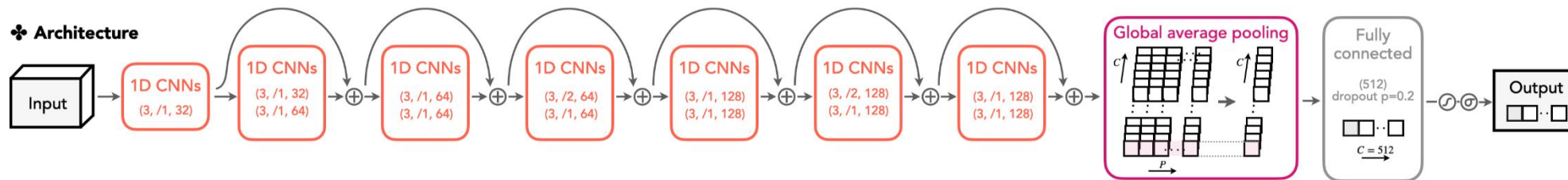
Supplementary

- DeepAK8 (1D CNN) Architecture

✦ Building block



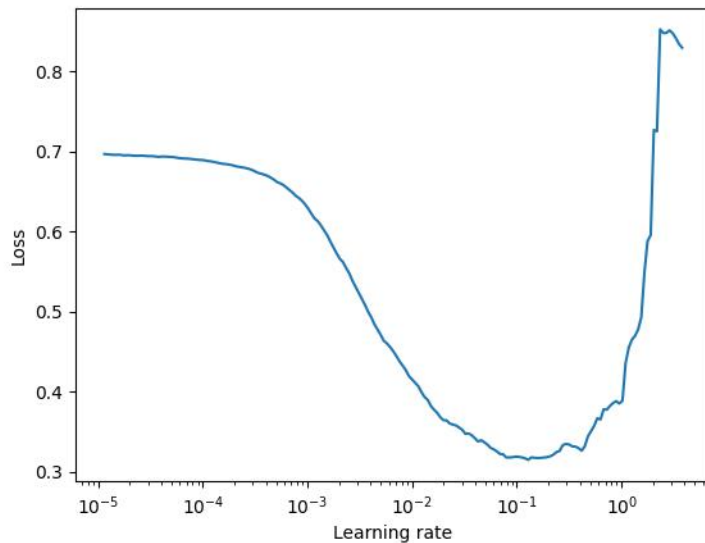
✦ Architecture



The full architecture of the DeepAK8 model, which is based on 1D CNN with ResNet architecture from [the documentation page](#).

Supplementary

- The optimizer
 - A wiser strategy: avoid the zig-zagging updating route, local minima, adapt different strategies for the fast-changing parameters and the slow ones
 - **Adam** (and its several variations): a widely used optimizer
 - **Ranger**: recently developed, advanced optimizer, combines RAdam and LookAhead

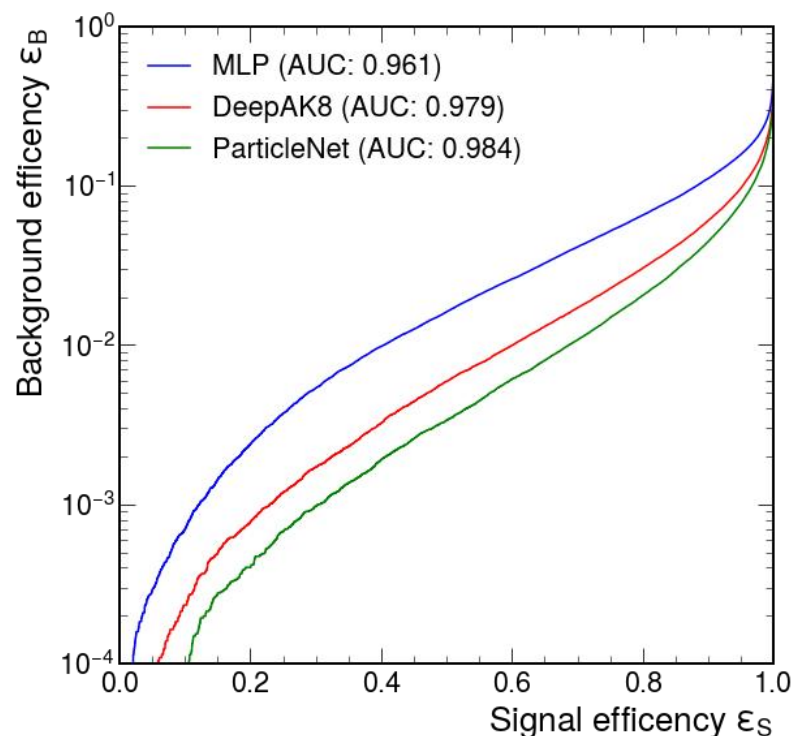


The result from LR finder by specifying optimizer AdamW case.

Supplementary

The ROC Curve

- Make the ROC curve comparing the performance of three trained models with the predicted root files.



Model	AUC	Accuracy	$1/e^B$ (@ $e^S=0.3$)
MLP	0.961	0.898	186
DeepAK8 (1D CNN)	0.979	0.927	585
ParticleNet (DGCNN)	0.984	0.936	1030

The ROC curve (background vs signal efficiency) and results for three models, using Ranger and start learning rate of 5e-3, from [the documentation page](#).